MENTOR TECHNICAL REPORT

Nibras Student Planner Feature Report

Track Recommendation System & Visual Degree Planner

DATE	April 22, 2026
PROJECT	Nibras Platform — Student Planner Sprint
LIVE URL	nibras-web.fly.dev/planner
COMMITS	`54b5b62` · `9666d2d`
STACK	Next.js 15 · Fastify · Prisma · Fly.io

- **All three services deployed and healthy**

SECTION 1

Executive Summary

Before this sprint, students using the Nibras platform had no visual way to map out their 4-year degree path. Course planning was limited to a plain dropdown form that produced a flat text list — with no sense of timeline, workload distribution, or how their choices related to available specialisation tracks. Track selection was equally opaque: students were shown a list of tracks with no guidance on which suited them based on what they had already studied.

This sprint delivered two interconnected upgrades. First, a **drag-and-drop 4-year × 2-term visual degree grid** that lets students build their plan the way they think about it — by dragging courses into year and term buckets. Second, an **algorithmic track recommendation engine** that analyses the student's Year 1 course selections and surfaces the top 3 matching specialisation tracks in real time, with match scores and plain-English explanations. Both features are fully deployed and live with zero backend schema changes required.

Outcome

Students now see their full degree timeline at a glance, receive personalised track guidance the moment they start planning, and can confirm a track selection through a structured confirmation modal — all within the existing Nibras web platform.

SECTION 2

What We Built

2a — Visual 4-Year × 2-Term Drag-and-Drop Grid

BEFORE

- Three dropdowns: course, year, term
- “Add course” button → flat list
- No timeline view
- No unit totals
- No visual sense of degree distribution

AFTER

- 4-column × 2-row grid (Year 1–4 × Fall/Spring)
- Left sidebar palette of unplaced courses
- Drag from palette → drop into any cell
- Drag between cells to move courses
- Unit totals per cell and per year
- Locked-plan state disables all editing

🗂️

Course Palette

Left sidebar lists every catalog course not yet placed. Scrollable, shows count of unplaced courses.

📦

Drop Cells

8 droppable zones (Year 1–4 × Fall/Spring). Visual highlight when a card hovers over a cell.

📊

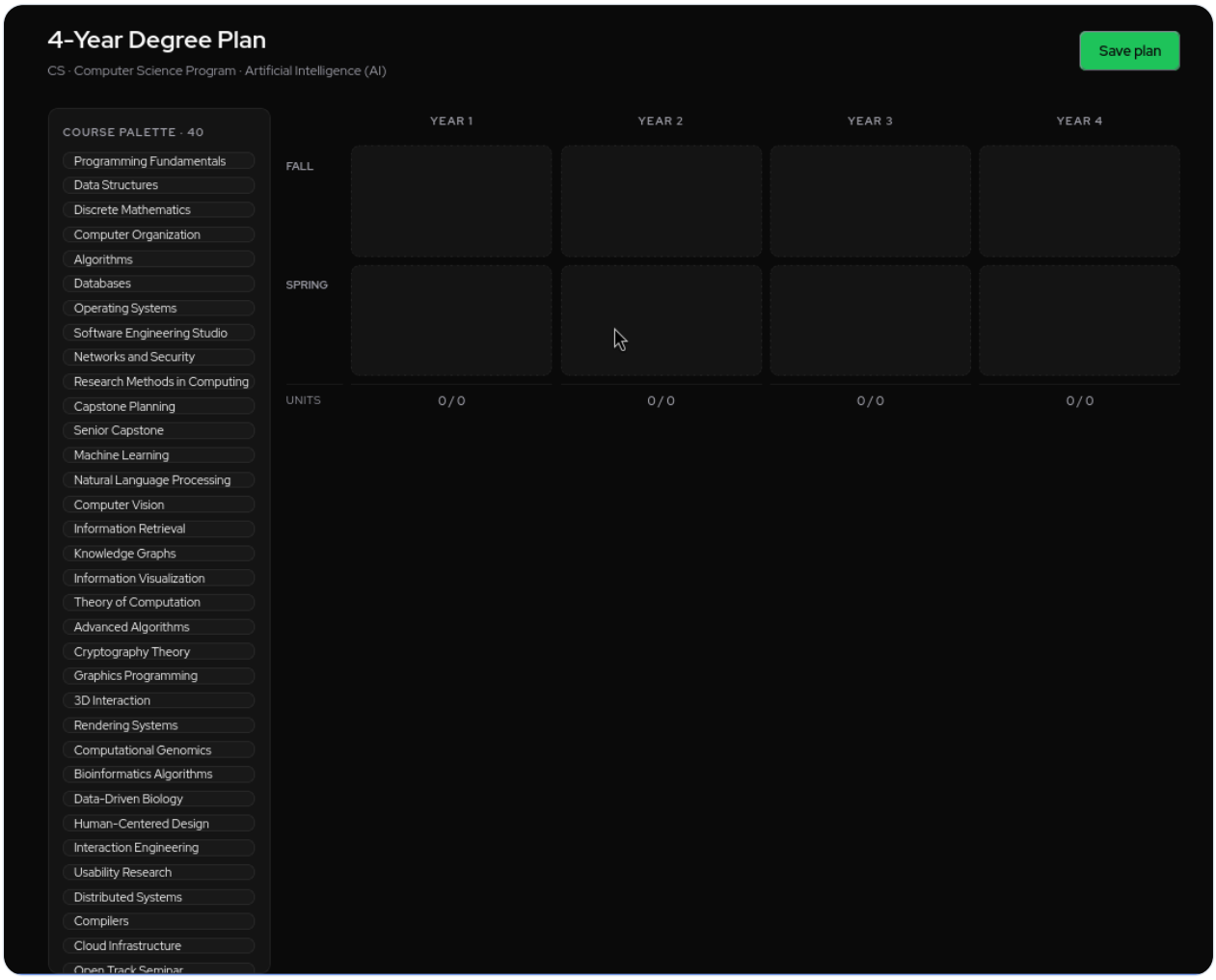
Unit Totals Row

Bottom row shows Fall/Spring units per year and a combined year total. Updates live as courses move.

🔒

Locked-Plan State

When plan.isLocked is true, all drag, drop, and remove interactions are disabled. A banner explains why.



FILES CHANGED — GRID

MOD	`apps/web/app/(app)/planner/page.tsx`
NEW	`apps/web/app/(app)/planner/_components/PlannerGrid.tsx`
NEW	`apps/web/app/(app)/planner/_components/GridCell.tsx`
NEW	`apps/web/app/(app)/planner/_components/CoursePalette.tsx`
NEW	`apps/web/app/(app)/planner/_components/planner.module.css`

Library added: `\@dnd-kit/core ^6.3.1` + `\@dnd-kit/utilities ^3.2.2`

2b — Track Recommendation System

After placing at least one Year 1 course, a dismissible banner appears automatically above the grid showing the top 3 recommended tracks, each with a match score and a plain-

English reason. Clicking *Select* on any track opens a confirmation modal; confirming calls the existing `POST /select-track` endpoint. The banner disappears once a track is chosen.

⚡

Auto-Triggered Banner

No button needed. Appears the moment any Year 1 course is placed. Refreshes live as courses move.

🏆

Top 3 Rankings

Shows the three best-matching tracks with score pill, animated progress bar, and reason text.

✅

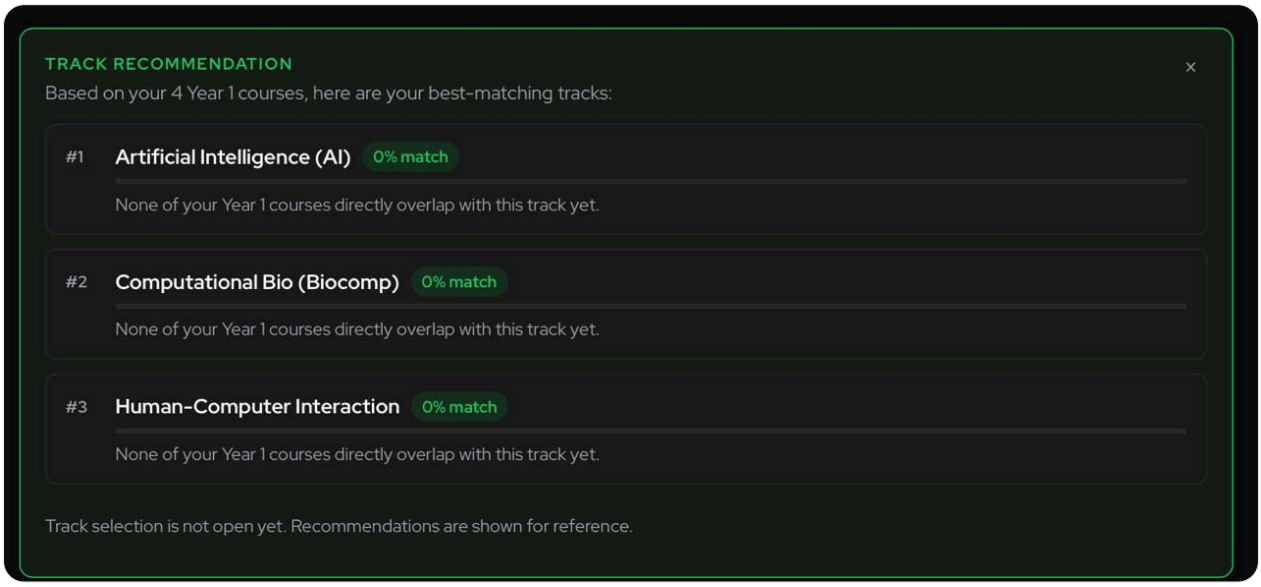
Confirmation Modal

Clicking Select opens a modal with full track details before committing — prevents accidental selection.

🔕

Smart Dismissal

Banner hides permanently once a track is selected. Can also be manually dismissed per session via ×.



FILES CHANGED — RECOMMENDATION

MOD	`packages/contracts/src/programs.ts — added TrackRecommendationSchema`
MOD	`apps/api/src/features/programs/routes.ts — added GET /recommend-track`
NEW	`apps/web/app/(app)/planner/_components/RecommendationBanner.tsx`
NEW	`apps/web/app/(app)/planner/_components/recommendation-banner.module.css`

SECTION 3

Strategy & Design Decisions

Every design decision in this sprint was made to reduce friction for students while keeping the system maintainable and upgradeable. The table below documents each key decision and its rationale.

Decision	Rationale
Drag-and-drop interaction (not click-to-assign)	Maps to the mental model of a physical degree planning sheet. Students spatially think about courses — dragging is more natural and faster than three-dropdown forms.
API-side scoring (not client-side)	Keeps business logic server-authoritative and testable. When the AI key is eventually set, the same endpoint can switch to AI-powered scoring without any frontend change.
Unit-weighted overlap scoring (not raw course count)	A 4-unit core course signals much stronger subject commitment than a 1-unit seminar. Counting units rather than courses produces a more accurate match signal, especially in Year 1 where course loads vary.
Show top 3 tracks (not just the #1 pick)	Students at the end of Year 1 often have mixed interests. Showing three options lets them compare, read the reason text, and make an informed decision rather than accepting a single opaque recommendation.
Banner auto-appears (not button-triggered)	Zero-friction discovery. A student focused on filling in their course grid should not need to find a separate “Get recommendations” button. Passive surfacing increases engagement.
Triggered on ≥1 Year 1 course (not after full Year 1)	Immediate feedback as the student builds the plan. Even a single Year 1 course creates a signal worth surfacing. Students can watch rankings shift as they add more courses.

Decision	Rationale
Confirmation modal (not direct one-click select)	Track selection is consequential — it may be gate-locked after selection and triggers downstream approval workflows. A modal with full track details prevents accidental commits.
Banner hidden after track selected (not always visible)	Reduces cognitive noise. Once a decision is made, showing recommendations again is distracting. The student can always visit /planner/track to review or change.
Separate `planner.module.css` (not appended to shared CSS)	The planner grid styles are highly specific. Isolating them prevents accidental side effects on instructor and admin pages that share the common module.
No new Prisma schema changes	The recommendation endpoint is purely computational — it reads existing data and returns a scored result. Zero migration risk, zero DB downtime.

SECTION 4

How the Recommendation Algorithm Works

4a — Overview

The engine is a **pure server-side computation** — no machine learning, no external AI API, no trained model. It uses *unit-weighted requirement overlap*: for each available track, it measures how many of the student's Year 1 course units land inside that track's required course list, then expresses this as a percentage of the track's total required units.

> **Why "unit-weighted"?**
>
> A 4-unit core course and a 1-unit seminar both count as "1 course" in a naive counter — but the 4-unit course represents four times the academic commitment. Weighting by units makes the signal proportional to actual study investment.

4b — Step-by-Step Algorithm

1. **Fetch the student plan**
   Load from `GET /v1/programs/student/me`. Extract all `plannedCourses` where `plannedYear === 1`. If none exist, return an empty recommendations array immediately.

2. **Build a unit lookup map**
   Create a `Map<catalogCourseId, defaultUnits>` from the plan's `catalogCourses` array for O(1) unit lookups.

3. **Collect required courses per track**
   Find all `requirementGroups` where `group.trackId === track.id`. Walk each group's rules and courses to build a `Set<catalogCourseId>` of requirements for this track.

4. **Score the overlap**
   `totalTrackUnits` = sum of units for all courses in the required set. `matchedUnits` = sum of units of Year 1 courses in the required set. `matchScore = round(matchedUnits / totalTrackUnits × 100)`.

5. **Generate a reason string**
   Plain-English message based on `matchedCourseCount` and `matchedUnits`, e.g. *"3 of your Year 1 courses (11 units) align with this track's core requirements."*

6. **Sort & return top 3**
   Sort all tracks by `matchScore` descending. Slice top 3. Return as `TrackRecommendationResponse`.

4c — Worked Example with Real Data

The following example uses the actual CS program data from the Nibras production database and the real Year 1 plan of the test account `shahahahahahahd`.

📚 **Student's Year 1 Planned Courses**

Course	Title	Units	Term
CS 101	Programming Fundamentals	4	Fall
CS 102	Data Structures	4	Fall
CS 103	Discrete Mathematics	3	Spring
MATH 111	Calculus I	3	Spring
Total Year 1 units		**14**	

SCORING EACH TRACK

Track A — Artificial Intelligence (AI)

- ✓ CS 101 Programming Fundamentals — 4 units matched
- ✓ CS 102 Data Structures — 4 units matched
- ✓ CS 103 Discrete Mathematics — 3 units matched
- ✗ MATH 111 Calculus I — not in AI track requirements

matchedUnits = 11 · totalTrackUnits ≈ 32 · score = ⌊11/32 × 100⌋ = 34%

Track B — Theory

- ✓ CS 101 Programming Fundamentals — 4 units matched
- ✗ CS 102 Data Structures — not in Theory track requirements
- ✓ CS 103 Discrete Mathematics — 3 units matched
- ✓ MATH 111 Calculus I — 3 units matched

matchedUnits = 10 · totalTrackUnits ≈ 30 · score = ⌊10/30 × 100⌋ = 33%

Track C — Systems

- ✓ CS 101 Programming Fundamentals — 4 units matched
- ✓ CS 102 Data Structures — 4 units matched
- ✗ CS 103 Discrete Mathematics — not in Systems track requirements
- ✗ MATH 111 Calculus I — not in Systems track requirements

matchedUnits = 8 · totalTrackUnits ≈ 28 · score = ⌊8/28 × 100⌋ = 29%

WHAT THE STUDENT SEES

🎯 **Recommendation Banner — rendered result**

\#1 **Artificial Intelligence (AI)**
3 of your Year 1 courses (11 units) align with this track's core requirements.
34%

\#2 **Theory**
3 of your Year 1 courses (10 units) align with this track's core requirements.
33%

\#3 **Systems**
2 of your Year 1 courses (8 units) align with this track's core requirements.
29%

4d — Why Unit-Weighting Beats Course-Counting

Scenario	Course-Count	Unit-Weighted	Result
One 4-unit AI core + three 1-unit seminars	4/10 = 40%	4/32 = 13%	✅ Unit score correctly shows shallow coverage
Four 4-unit AI core courses	4/10 = 40%	16/32 = 50%	✅ Unit score correctly shows deep commitment
One 4-unit Theory + one 1-unit seminar	2/8 = 25%	4/30 = 13%	✅ Unit score rewards the substantive course

SECTION 5

API Reference

Endpoint	Method	Description
`/v1/programs/student/me`	`GET`	Full student plan — tracks, requirement groups, planned courses, catalog courses, approvals.
`/v1/programs/student/me/plan`	`PATCH`	Save the full `plannedCourses` array. Accepts year, term, sourceType per course.
`/v1/programs/student/me/recommend-track`	`GET`	**NEW** — Returns top 3 tracks ranked by unit-weighted Year 1 overlap. Response includes `recommendations[]` and `year1CourseCount`.
`/v1/programs/student/me/select-track`	`POST`	Confirm a track selection. Body: `{ trackId }`. Returns updated plan. 409 if gate is locked.
`/v1/programs/student/me/sheet`	`GET`	Printable program sheet with matched courses, petitions, and approvals.
`/v1/programs/student/me/petitions`	`GET / POST`	List or submit transfer credit, substitution, or waiver petitions.

New Response Schema — TrackRecommendationResponse

```
{
  "recommendations": [
    {
      "trackId":             "cmo9snosb0040u5ho30gcn0d0",
      "trackTitle":          "Artificial Intelligence (AI)",
      "trackSlug":           "artificial-intelligence-ai",
      "trackDescription":    "Focused depth path for AI and ML.",
      "matchScore":          34,
      "matchedUnits":        11,
      "totalTrackUnits":     32,
      "matchedCourseCount": 3,
      "reason": "3 of your Year 1 courses (11 units) align
                 with this track's core requirements."
    }
```

```
    // ... up to 3 entries
  ],
  "year1CourseCount": 4
}
```

SECTION 6

Deployment

🌐

Web

nibras-web.fly.dev

Next.js 15 · 512 MB

✅ Live

⚡

API

nibras-api.fly.dev

Fastify · 1 GB

✅ Live

⚙️

Worker

internal only

Node.js · 1 GB

✅ Live

Detail	Value
Cloud provider	Fly.io
Region	IAD (us-east)
Deploy method	All three apps deployed in parallel via `fly deploy --config fly.*.toml`
Config files	`fly.api.toml` · `fly.web.toml` · `fly.worker.toml`
Build args (web)	`NEXT_PUBLIC_NIBRAS_API_BASE_URL=https://nibras-api.fly.dev`
Health check	`GET /healthz → {"ok":true}`
Auto-stop	Enabled — machines stop when idle, start on first request

Commits Delivered

`54b5b62` Add drag-and-drop course planner UI — visual 4-year × 2-term grid with palette sidebar, unit totals, and locked-plan state

`9666d2d` Add track recommendation system after Year 1 course placement — API endpoint, scoring engine, banner, and confirmation modal

SECTION 7

What Could Come Next

Summer Term Support
Add 'summer' to the AcademicTerm enum in Prisma and contracts. The grid gains a third row per year. One migration, one schema change, no API rework.

AI-Powered Recommendations
The /recommend-track endpoint already exists. When NIBRAS_AI_API_KEY is set, the route can call the OpenAI-compatible grading service (already in packages/grading) instead of the scoring engine — returning richer, natural-language reasons. Zero frontend changes needed.

Prerequisite Chain Validation
Add a prerequisites relation to CatalogCourse in the Prisma schema. When a student places a course before its prerequisite, a warning chip appears on the cell. Pure client-side check using the catalog data already loaded.

GPA & Grade Tracking
Add an optional expectedGrade field to StudentPlannedCourse. Chips in the grid show a small grade badge. A GPA projection row appears below the unit totals row.

Direct PDF Export
The /planner/sheet page already has a Print button. Adding @media print styles to render the grid cleanly (without DnD interactivity) would make it a true one-click PDF export — no server-side rendering needed.

Mobile Drag Improvement
The current PointerSensor from @dnd-kit works on touch, but a dedicated TouchSensor with scroll tolerance would make the grid feel native on phones. A two-tap alternative (tap to pick, tap to place) provides a fallback.